



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ	Информатика и системы управления (ИУ)
КАФЕДРА	Информационная безопасность (ИУ8)
ДИСЦИПЛИНА	Математические основы искусственного интеллекта

ДОМАШНЕЕ ЗАДАНИЕ №2.2

Необходимые и достаточные условия существования условного экстремума
название

Группа ИУ8-13М

Студент	5 вариант		Никулина З.Е.
		<i>подпись</i>	<i>фамилия, и.о.</i>

Преподаватель		Каганов Ю.Т.
	<i>подпись</i>	<i>фамилия, и.о.</i>

Москва, 2025 г.

Цель работы – исследование необходимых и достаточных условий существования экстремума функции с учетом ограничений (условный экстремум), вычисление экстремумов функции.

Практическая часть

1 Постановка задачи

Дважды непрерывно дифференцируемые целевая функция $f(x)$ и функции ограничений $g_j(x) = g_j(x_1, \dots, x_n) = 0, j = \overline{1, m}; g_j(x) \leq 0, j = m + 1, \dots, p$, определяющие множество допустимых решений $X \in R^n$. Требуется исследовать функцию на экстремум, т.е. определить точки $x^e \in R^n$ ее локальных минимумов и максимумов на X :

$$f(x^e) = \min_{x \in X} f(x) = \max_{x \in X} f(x), \quad (1)$$

$$\text{где } X = \left\{ x \mid \begin{array}{l} g_j(x) = 0, j = \overline{1, m}; m < n \\ g_j(x) \leq 0, j = m + 1, \dots, p \end{array} \right\}.$$

2 Исследование экстремумов

Необходимо выявить экстремумы для функции:

$$\left\{ \begin{array}{l} f(x) = 4x_1^2 + 4x_1 + x_2^2 - 8x_2 + 5 \rightarrow \text{extr} \\ g_1(x) = 2x_1 + x_2 - 6 \leq 0, \\ g_2(x) = -x_1 \leq 0, \\ g_3(x) = -x_2 \leq 0. \end{array} \right.$$

На рисунке 1 изображены необходимые условия экстремума 1-го порядка.

Обобщённая функция Лагранжа:

$$L(x, \lambda_0, \lambda) = \lambda_0 * f(x) + \lambda_1 * g_1(x) + \lambda_2 * g_2(x) + \lambda_3 * g_3(x)$$

Необходимые условия 1-го порядка

$$\text{Стационарность: } dL/dx_i = 0$$

$$\text{Допустимость: } g(x) \leq 0. \lambda_j * g_j(x) = 0.$$

Рисунок 1 – Необходимые условия экстремума

На рисунке 2 показано решение системы уравнений для двух случаев. Решения дают условно-стационарные точки. Точки соответствуют вершинам многоугольник ограничений.

```

Решение систем для  $\lambda_0=0$  (вырожденный) и  $\lambda_0 \neq 0$  (классический).
 $\lambda_0=0$  (вырожденный), минимум: найдено 3 решения(й).
    Активные (0, 1),  $x=(\text{np.float64}(0.0), \text{np.float64}(6.0))$ ,  $\lambda=(\text{np.float64}(0.0), \text{np.float64}(0.0), \text{np.float64}(0.0))$ ,  $f=-7.000000$ 
    Активные (0, 2),  $x=(\text{np.float64}(3.0), \text{np.float64}(-0.0))$ ,  $\lambda=(\text{np.float64}(0.0), \text{np.float64}(0.0), \text{np.float64}(-0.0))$ ,  $f=53.000000$ 
    Активные (1, 2),  $x=(\text{np.float64}(-0.0), \text{np.float64}(-0.0))$ ,  $\lambda=(\text{np.float64}(0.0), \text{np.float64}(-0.0), \text{np.float64}(-0.0))$ ,  $f=5.000000$ 
 $\lambda_0=0$  (вырожденный), максимум: найдено 3 решения(й).
    Активные (0, 1),  $x=(\text{np.float64}(0.0), \text{np.float64}(6.0))$ ,  $\lambda=(\text{np.float64}(0.0), \text{np.float64}(0.0), \text{np.float64}(0.0))$ ,  $f=-7.000000$ 
    Активные (0, 2),  $x=(\text{np.float64}(3.0), \text{np.float64}(-0.0))$ ,  $\lambda=(\text{np.float64}(0.0), \text{np.float64}(0.0), \text{np.float64}(-0.0))$ ,  $f=53.000000$ 
    Активные (1, 2),  $x=(\text{np.float64}(-0.0), \text{np.float64}(-0.0))$ ,  $\lambda=(\text{np.float64}(0.0), \text{np.float64}(-0.0), \text{np.float64}(-0.0))$ ,  $f=5.000000$ 
 $\lambda_0 \neq 0$  (классический), минимум: найдено 1 решения(й).
    Активные (1, ),  $x=(\text{np.float64}(0.0), \text{np.float64}(4.0))$ ,  $\lambda=(\text{np.float64}(0.0), \text{np.float64}(4.0), \text{np.float64}(0.0))$ ,  $f=-11.000000$ 
 $\lambda_0 \neq 0$  (классический), максимум: найдено 3 решения(й).
    Активные (0, ),  $x=(\text{np.float64}(0.25), \text{np.float64}(5.5))$ ,  $\lambda=(\text{np.float64}(-3.0), \text{np.float64}(0.0), \text{np.float64}(0.0))$ ,  $f=-7.500000$ 
    Активные (0, 1),  $x=(\text{np.float64}(0.0), \text{np.float64}(6.0))$ ,  $\lambda=(\text{np.float64}(-4.0), \text{np.float64}(-4.0), \text{np.float64}(0.0))$ ,  $f=-7.000000$ 
    Активные (0, 2),  $x=(\text{np.float64}(3.0), \text{np.float64}(0.0))$ ,  $\lambda=(\text{np.float64}(-14.0), \text{np.float64}(0.0), \text{np.float64}(-22.0))$ ,  $f=53.000000$ 

```

Рисунок 2 – Решение системы уравнений для двух случаев

Проверка достаточных условий была проведена программой, результаты показаны на рисунке 3. Для всех стационарных точек была выполнена проверка по первому и второму порядку. Для вершин (0,6), (3,0), (0,0) достаточные условия первого порядка оказались неопределёнными, поэтому дополнительно исследовались касательные направления: векторы. На этих направлениях квадратичная форма положительна, что подтверждает наличие минимума. Однако истинным глобальным минимумом оказался (0,4) с ($f=-11$), где активно только ограничение g_2 , и проверка второго порядка показала положительность квадратичной формы — это строгое условие локального минимума. Для кандидатов на максимум условия второго порядка не подтвердили экстремум, кроме точки (3,0), где значение функции максимально ($f=53$). Таким образом, окончательный вывод: глобальный минимум в точке (0,4) со значением -11, глобальный максимум в точке (3,0) со значением 53.

```

– Кандидаты на минимум (после шага 3):
x=(np.float64(0.0), np.float64(6.0)), f=-7.000000, λ=(np.float64(0.0), np.float64(0.0), n
p.float64(0.0)), активны [0, 1]
Достат. 1-го порядка: неопределён (нужно 2-й порядок)
Касательные направления (для  $\lambda \geq 0$ ): [array([1., 0.]), array([0., 1.])]
Значения dTHd на базе: [8.0, 2.0], вывод: минимум
x=(np.float64(3.0), np.float64(-0.0)), f=53.000000, λ=(np.float64(0.0), np.float64(0.0), n
p.float64(-0.0)), активны [0, 2]
Достат. 1-го порядка: неопределён (нужно 2-й порядок)
Касательные направления (для  $\lambda \geq 0$ ): [array([1., 0.]), array([0., 1.])]
Значения dTHd на базе: [8.0, 2.0], вывод: минимум
x=(np.float64(-0.0), np.float64(-0.0)), f=5.000000, λ=(np.float64(0.0), np.float64(-0.0),
np.float64(-0.0)), активны [1, 2]
Достат. 1-го порядка: неопределён (нужно 2-й порядок)
Касательные направления (для  $\lambda \geq 0$ ): [array([1., 0.]), array([0., 1.])]
Значения dTHd на базе: [8.0, 2.0], вывод: минимум
x=(np.float64(0.0), np.float64(4.0)), f=-11.000000, λ=(np.float64(0.0), np.float64(4.0), n
p.float64(0.0)), активны [1]
Достат. 1-го порядка: неопределён (нужно 2-й порядок)
Касательные направления (для  $\lambda \geq 0$ ): [array([0., 1.])]
Значения dTHd на базе: [2.0], вывод: минимум
⇒ Итог: x*=(np.float64(0.0), np.float64(4.0)), f*=-11.000000

– Кандидаты на максимум (после шага 3):
x=(np.float64(0.0), np.float64(6.0)), f=-7.000000, λ=(np.float64(0.0), np.float64(0.0), n
p.float64(0.0)), активны [0, 1]
Достат. 1-го порядка: неопределён (нужно 2-й порядок)
Касательные направления (для  $\lambda \geq 0$ ): [array([1., 0.]), array([0., 1.])]
Значения dTHd на базе: [8.0, 2.0], вывод: нет вывода
x=(np.float64(3.0), np.float64(-0.0)), f=53.000000, λ=(np.float64(0.0), np.float64(0.0), n
p.float64(-0.0)), активны [0, 2]
Достат. 1-го порядка: неопределён (нужно 2-й порядок)
Касательные направления (для  $\lambda \geq 0$ ): [array([1., 0.]), array([0., 1.])]
Значения dTHd на базе: [8.0, 2.0], вывод: нет вывода
x=(np.float64(-0.0), np.float64(-0.0)), f=5.000000, λ=(np.float64(0.0), np.float64(-0.0),
np.float64(-0.0)), активны [1, 2]
Достат. 1-го порядка: неопределён (нужно 2-й порядок)
Касательные направления (для  $\lambda \geq 0$ ): [array([1., 0.]), array([0., 1.])]
Значения dTHd на базе: [8.0, 2.0], вывод: нет вывода
x=(np.float64(0.25), np.float64(5.5)), f=-7.500000, λ=(np.float64(-3.0), np.float64(0.0),
np.float64(0.0)), активны [0]
Достат. 1-го порядка: неопределён (нужно 2-й порядок)
Касательные направления (для  $\lambda \geq 0$ ): [array([ 1., -2.])]
Значения dTHd на базе: [16.0], вывод: нет вывода
Не определён.

```

Рисунок 3 – Проверка достаточных условий

Графическое изображения решения показано на рисунке 4, поверхность функции изображена на рисунке 5. Глобальный минимум функция = -11 достигается в точке (0,4), глобальный максимум =53 – в точке (3,0).

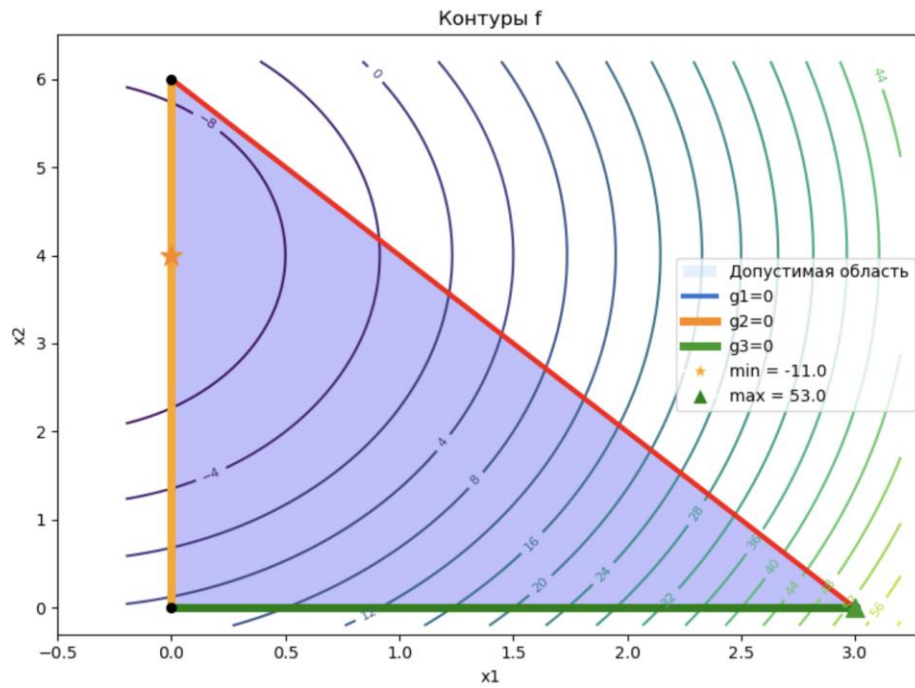


Рисунок 4 – Допустимая область решения

Поверхность $f(x_1, x_2)$

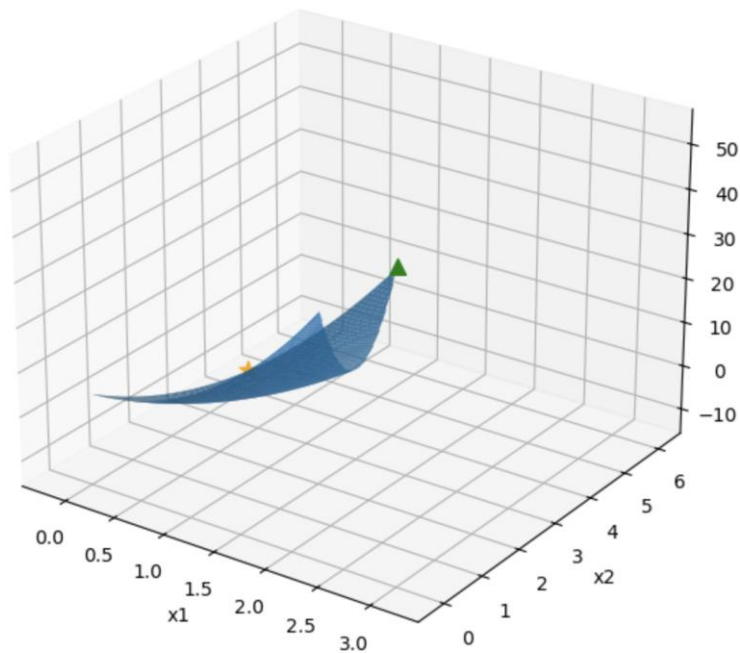


Рисунок 5 – Поверхность функции

На рисунке 6 показана поверхность с углом обзора 150. График наглядно демонстрирует выпуклую форму функции и то, как ограничения «обрезают» поверхность, оставляя треугольную область, внутри которой и достигаются найденные значения минимума и максимума.

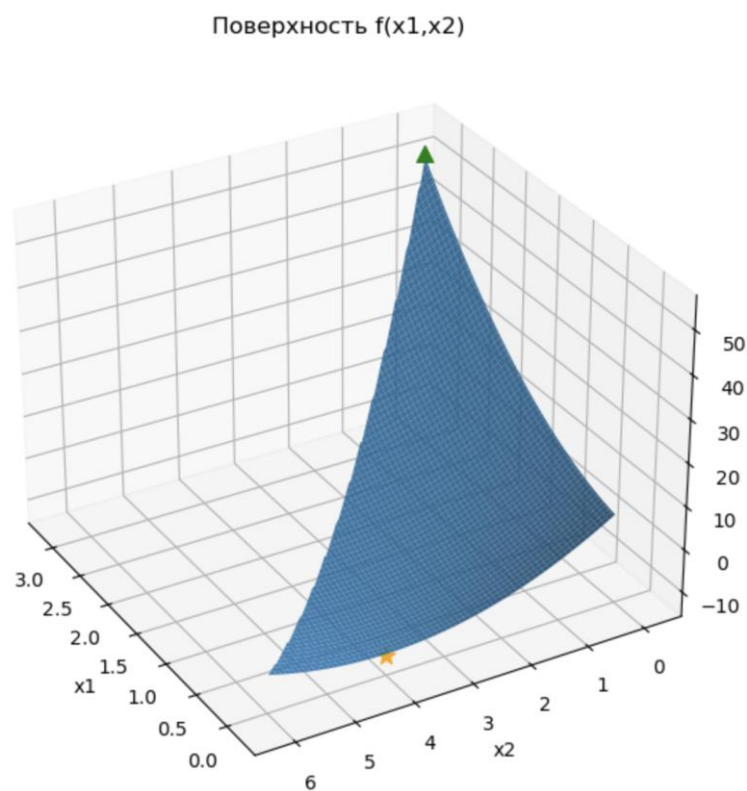


Рисунок 6 – Поверхность функции для другого угла обзора
Листинги программ приведены в Приложении А.

ПРИЛОЖЕНИЕ А

Листинги программ

```
import numpy as np
import itertools as it
import math
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
np.set_printoptions(precision=6, suppress=True)

def f(x):
    x1, x2 = x
    return 4*x1**2 + 4*x1 + x2**2 - 8*x2 + 5

def grad_f(x):
    x1, x2 = x
    return np.array([8*x1 + 4, 2*x2 - 8], dtype=float)

H = np.array([[8.0, 0.0],
              [0.0, 2.0]])

g = [
    lambda x: 2*x[0] + x[1] - 6, # g1<=0
    lambda x: -x[0],           # g2<=0; x1>=0
    lambda x: -x[1],           # g3<=0; x2>=0
]

grads_g = [
    np.array([2.0, 1.0]),
    np.array([-1.0, 0.0]),
    np.array([0.0, -1.0]),
]

names_g = ["g1: 2x1+x2-6<=0", "g2: -x1<=0", "g3: -x2<=0"]

print("Обобщённая функция Лагранжа:")
print("L(x, λ0, λ) = λ0*f(x) + λ1*g1(x) + λ2*g2(x) + λ3*g3(x)")

def feasible(x, tol=1e-9):
    return all(gi(x) <= tol for gi in g)

def active_set(x, tol=1e-9):
    A = [j for j, gj in enumerate(g) if abs(gj(x)) <= tol]
    return A

def solve_stationarity(A, sign, lam0=1.0):
    m = 2 + len(A)
    A_mat = np.zeros((2+len(A), m))
    b_vec = np.zeros(2+len(A))
    A_mat[0:2, 0:2] = lam0 * H
    offset = 2
    for k, j in enumerate(A):
        A_mat[0:2, offset+k] = grads_g[j]
    b_vec[0:2] = -lam0 * np.array([4.0, -8.0])

    for r, j in enumerate(A):
        if j == 0:
            # 2x1 + x2 - 6 = 0
            A_mat[2+r, 0] = 2.0
            A_mat[2+r, 1] = 1.0
```

```

        b_vec[2+r] = 6.0
    elif j == 1:
        # -x1 = 0
        A_mat[2+r, 0] = -1.0
        b_vec[2+r] = 0.0
    elif j == 2:
        # -x2 = 0
        A_mat[2+r, 1] = -1.0
        b_vec[2+r] = 0.0
try:
    sol = np.linalg.solve(A_mat, b_vec)
except np.linalg.LinAlgError:
    return None

x = sol[0:2]
lam = np.zeros(3)
for k, j in enumerate(A):
    lam[j] = sol[2+k]

for j in range(3):
    if j not in A:
        lam[j] = 0.0

if not feasible(x):
    return None
ok_signs = all((lam[j] >= -1e-10 if sign > 0 else lam[j] <= 1e-10) for j in A)
if not ok_signs:
    return None
return x, lam

print("Необходимые условия 1-го порядка")
print("Стационарность: dL/dxi = 0")
print("Допустимость: g(x) ≤ 0. λj * gj(x) = 0.")
print("Знаки λj: для минимума λj ≥ 0, для максимума λj ≤ 0.")

print("Решение систем для λ0=0 (вырожденный) и λ0≠0 (классический).")

candidates = {"min": [], "max": []}
for case, lam0 in [("λ0=0 (вырожденный)", 0.0), ("λ0≠0 (классический)", 1.0)]:
    for sign, tag in [(+1, "min"), (-1, "max")]:
        found = []
        for r in range(0, 4):
            for A in it.combinations(range(3), r):
                sol = solve_stationarity(list(A), sign, lam0=lam0)
                if sol is None:
                    continue
                x, lam = sol
                found.append((tuple(sorted(A)), tuple(np.round(x,6)), tuple(np.round(lam,6)), f(x)))
        if found:
            uniq = {}
            for A,x,lam,val in found:
                uniq[x] = (A,lam,val)
            print(f" {case}, {{('минимум' if sign>0 else 'максимум')}}: найдено {len(uniq)} решения(й).")
            for x, (A,lam,val) in uniq.items():
                print(f"    Активные {A}, x={x}, λ={lam}, f={val:.6f}")
                candidates[tag].append({"lam0": lam0, "A":A, "x":np.array(x,float), "lam":np.array(lam,float), "f":val})
        else:
            print(f" {case}, {{('минимум' if sign>0 else 'максимум')}}: решений нет.")
print()

```

```

def uniq_by_x(arr):
    keep = {}
    for d in arr:
        key = tuple(np.round(d["x"],8))
        if key not in keep:
            keep[key]=d
    return list(keep.values())

candidates["min"] = uniq_by_x(candidates["min"])
candidates["max"] = uniq_by_x(candidates["max"])

print("Проверка достаточных условий.")
def first_order_sufficient(x, lam, sign):
    A = active_set(x)
    l = len(A)
    n = 2
    if l == n:
        if sign>0 and all(lam[j]>0 for j in A): return "локальный минимум по 1-му порядку"
        if sign<0 and all(lam[j]<0 for j in A): return "локальный максимум по 1-му порядку"
    return "неопределён (нужно 2-й порядок)"

def second_order_tests(x, lam, sign):
    A = active_set(x)
    Aj_pos = [j for j in A if (lam[j]>0 if sign>0 else lam[j]<0)]
    Aj_zero = [j for j in A if abs(lam[j])<=1e-10]
    Apos = np.array([grads_g[j] for j in Aj_pos], dtype=float)
    if Apos.size==0:
        basis = [np.array([1.0,0.0]), np.array([0.0,1.0])]
    elif Apos.shape[0]==1:
        a = Apos[0]
        if abs(a[1])>1e-12:
            basis = [np.array([1.0, -a[0]/a[1]])]
        else:
            basis = [np.array([0.0,1.0])]
    else:
        basis = []
    def quad_on_basis():
        vals = []
        if not basis:
            return [0.0]
        for v in basis:
            vals.append(float(v.T@H@v))
        return vals
    quad_vals = quad_on_basis()
    if sign>0:
        ok_quad = all(v>=1e-12 for v in quad_vals)
    else:
        ok_quad = all(v<=-1e-12 for v in quad_vals)
    second = quad_vals

    return {
        "A_pos": Aj_pos, "A_zero": Aj_zero,
        "tangent_basis": basis,
        "quad_vals_on_basis": quad_vals,
        "passes_second_order": ok_quad,
        "conclusion": ("минимум" if (sign>0 and ok_quad) else ("максимум" if (sign<0
and ok_quad) else "нет вывода"))
    }

```

```

def report(tag, items, sign):
    best = None
    print(f"\n— Кандидаты на {'минимум' if sign>0 else 'максимум'} (после шага 3):")
    for d in items:
        x, lam = d["x"], d["lam"]
        print(f"  x={tuple(np.round(x,6))}, f={f(x):.6f}, λ={tuple(np.round(lam,6))},
активны {active_set(x)}")
        print(f"    Достат. 1-го порядка:", first_order_sufficient(x,lam,sign))
        s2 = second_order_tests(x,lam,sign)
        print(f"    Касательные направления (для λ≥0): {s2['tangent_basis']}")
        print(f"    Значения dTHd на базе: {s2['quad_vals_on_basis']}, вывод: {s2['con-
clusion']}")
        if s2["conclusion"] in ["минимум","максимум"]:
            if best is None: best = d
            else:
                if (sign>0 and f(x)<f(best['x'])) or (sign<0 and f(x)>f(best['x'])):
                    best = d
    if best:
        print(f"  ⇒ Итог: x*={tuple(np.round(best['x'],6))}, f*={f(best['x']):.6f}")
    else:
        print("Не определён.")

report("min", candidates["min"], +1)
report("max", candidates["max"], -1)

xmin = np.array([0.0,4.0])
xmax = np.array([3.0,0.0])
print("Значения функции в точках экстремума:")
print(f"  Минимум: x*={tuple(xmin)}, f(x*)={f(xmin)}")
print(f"  Максимум: x̂={tuple(xmax)}, f(x̂)={f(xmax)}")

def f_val(x1, x2):
    return 4*x1**2 + 4*x1 + x2**2 - 8*x2 + 5

xmin = np.array([0.0, 4.0])
xmax = np.array([3.0, 0.0])

x1 = np.linspace(-0.2, 3.2, 240)
x2 = np.linspace(-0.2, 6.2, 240)
X1, X2 = np.meshgrid(x1, x2)
Z = f_val(X1, X2)

feas = (2*X1 + X2 - 6 <= 1e-12) & (X1 >= -1e-12) & (X2 >= -1e-12)

plt.figure(figsize=(8,6))
cs = plt.contour(X1, X2, Z, levels=20)
plt.clabel(cs, inline=1, fontsize=8)

plt.contourf(X1, X2, feas.astype(int), levels=[0.5, 1.5], alpha=0.25, colors="blue")

xx = np.linspace(0, 3, 200)
ln1, = plt.plot(xx, 6 - 2*xx, color="red", linewidth=3, label="g1=0")
ln2, = plt.plot([0, 0], [0, 6], color="orange", linewidth=5, label="g2=0")
ln3, = plt.plot([0, 3], [0, 0], color="green", linewidth=5, label="g3=0")

verts = np.array([[0,0],[0,6],[3,0]])
plt.scatter(verts[:,0], verts[:,1], color="black", s=30, zorder=5)

plt.scatter([xmin[0]], [xmin[1]], marker="*", s=180, color="#ff8800", zorder=6, la-
bel="min -11.0")
plt.scatter([xmax[0]], [xmax[1]], marker="^", s=120, color="#2E9B1F", zorder=6, la-
bel="max 53.0")

```

```

from matplotlib.patches import Patch
from matplotlib.lines import Line2D
legend_elems = [
    Patch(facecolor="#94c6ff", edgecolor="none", alpha=0.25, label="Допустимая
область"),
    Line2D([0],[0], color="#2A76D2", lw=3, label="g1=0"),
    Line2D([0],[0], color="#ff8800", lw=5, label="g2=0"),
    Line2D([0],[0], color="#2E9B1F", lw=5, label="g3=0"),
    Line2D([0],[0], marker="*", color="w", markerfacecolor="orange", markersize=10, la-
bel="min = -11.0"),
    Line2D([0],[0], marker="^", color="w", markerfacecolor="green", markersize=10, la-
bel="max = 53.0"),
]
plt.legend(handles=legend_elems, loc="best", frameon=True)

plt.title("Контуры f")
plt.xlabel("x1")
plt.ylabel("x2")
plt.xlim(-0.5, 3.3)
plt.ylim(-0.3, 6.5)
plt.tight_layout()
plt.show()
fig = plt.figure(figsize=(6,6))
ax = fig.add_subplot(111, projection='3d')

Zs = Z.copy(); Zs[~feas] = np.nan
surf = ax.plot_surface(X1, X2, Zs, rstride=3, cstride=3, linewidth=0.2, antiali-
ased=True, alpha=0.9)

ax.scatter(xmin[0], xmin[1], f_val(*xmin), s=90, marker="*", color="orange", la-
bel="min")
ax.scatter(xmax[0], xmax[1], f_val(*xmax), s=70, marker="^", color="green", la-
bel="max")

ax.set_title("Поверхность f(x1,x2)")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("f")
ax.view_init(elev=25, azim=-55)
plt.tight_layout()
plt.show()
fig = plt.figure(figsize=(6,6))
ax = fig.add_subplot(111, projection='3d')

Zs = Z.copy(); Zs[~feas] = np.nan
surf = ax.plot_surface(X1, X2, Zs, rstride=3, cstride=3, linewidth=0.2, antiali-
ased=True, alpha=0.9)

ax.scatter(xmin[0], xmin[1], f_val(*xmin), s=90, marker="*", color="orange", la-
bel="min")
ax.scatter(xmax[0], xmax[1], f_val(*xmax), s=70, marker="^", color="green", la-
bel="max")

ax.set_title("Поверхность f(x1,x2)")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("f")
ax.view_init(elev=30, azim=150)
plt.tight_layout()
plt.show()

```